Software Engineering Research Group
MSc Thesis Style

Version of April 20, 2026

Leon Moonen

Software Engineering Research Group MSc Thesis Style

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Leon Moonen
born in Weert, the Netherlands

Software Engineering Research Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

Some Company
With it's address
ThePlace, the Netherlands
www.url.nl

© 2026 Leon Moonen. *Note that this notice is for demonstration purposes and that the LaTeX style and document source are free to use as basis for your MSc thesis.*

Cover picture: A “random” maze generated in postscript.

Software Engineering Research Group MSc Thesis Style

Author: Leon Moonen
Student id: 123456789
Email: Leon.Moonen@computer.org

Abstract

This document describes the standard thesis style for the Software Engineering department at Delft University of Technology. The document and it's source are an example of the use of the standard LaTeX style file. In addition the final appendix to this document contains a number of requirements and guidelines for writing a Software Engineering MSc thesis.

Your thesis should either employ this style or follow it closely.

Thesis Committee:

Chair:	Prof. Dr. A. van Deursen, Faculty EEMCS, TU Delft
University supervisor:	Dr. E. Visser, Faculty EEMCS, TU Delft
Company supervisor:	Drs. E.X. Ternal, Some Company
Committee Member:	Dr. S.T.A.F.F. Member, Faculty EEMCS, TU Delft

Preface

This is where you thank people for helping you etc.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus massa pede, feugiat sit amet, mollis in, sodales at, augue. Ut sit amet nisi egestas risus consequat adipiscing. Nulla non diam. Proin volutpat, lacus quis volutpat scelerisque, leo urna rhoncus arcu, vel ultrices dui lacus id lorem. Nam pulvinar adipiscing odio. Etiam tellus lorem, malesuada in, scelerisque sit amet, consequat a, tellus. Curabitur non urna. Mauris facilisis tempor nulla. Nam euismod semper massa. Nullam id nulla. Duis mattis nunc ut ipsum. Proin libero purus, posuere ut, tincidunt sit amet, accumsan sit amet, nisl. Integer commodo. Pellentesque suscipit, diam vel bibendum interdum, magna mauris venenatis lorem, vitae tristique nibh lacus convallis velit. Sed tellus. Mauris placerat lectus ut tellus rutrum blandit. Aliquam erat volutpat.

Suspendisse potenti. Proin sodales eros non lacus. Nam magna sapien, tristique ut, hendrerit ultricies, pretium ut, ante. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Nulla facilisi. In libero risus, pellentesque vitae, interdum id, tincidunt ut, sapien. Mauris nec massa sit amet leo dictum pretium. Curabitur iaculis euismod mauris. Donec diam sem, pulvinar at, luctus id, blandit nec, pede. Nam scelerisque sollicitudin nunc. Nam malesuada mauris id ligula. Donec suscipit posuere justo. Mauris sed libero in mi nonummy tincidunt.

Leon Moonen
Delft, the Netherlands
April 20, 2026

Contents

Preface **iii**

Contents **v**

List of Figures **vii**

1 Introduction **1**
1.1 Research Question(s) 1

2 Background & Theory **3**
2.1 From standard machine learning to Operator Learning 3
2.2 Fourier Neural Operator 3
2.3 Convolutional Neural Operator 4
2.4 Mesh-free Operator Learning and geo-FNO 4
2.5 Discretization invariance on mesh-free grids 5
2.6 Operator Learning 5
2.7 Fourier neural operator 9
2.8 Convolutional neural operator 9
2.9 Mesh-free Operator Learning and geo-FNO 11
2.10 Discretization invariance on mesh-free grids 13

3 Experimental Setup **15**
3.1 Datasets 15
3.2 Training setup 20
3.3 Uniform-grid resolution transfer experiments 20
3.4 Non-uniform-grid geo-FNO evaluations 21

4 Uniform grids - Results and Discussion **23**
4.1 results 23
4.2 Discussion/Interpretation/Analysis 26

5 Non-uniform meshes - Results and Discussion **29**
5.1 Results 29
5.2 Discussion/Interpretation/Analysis 30

6 Conclusions and Future Work **33**
6.1 Contributions and Conclusions 33
6.2 Future work 33

Bibliography **35**

A Disclosure on AI use **37**

B Navier-Stokes data generation details **39**

C HTGR input engineering details **41**

D Hyperparameters **43**

E Geo-FNO bug fix **45**

F Requirements and Guidelines **47**
F.1 Requirements 47
F.2 Guidelines 48

List of Figures

2.1 an illustration of aliasing. The blue curve represents a high-frequency signal, while the red curve is a lower-frequency function that passes through the same sampled points (black dots). Since both functions agree at the sampling locations, the discrete representation cannot distinguish between them. 6

3.1 Initial and final vorticity fields for a sample from the Navier-Stokes dataset. . . 16

3.2 Example of an input diffusion coefficient field $a(x,y)$ and the resulting pressure field $u(x,y)$ with Dirichlet boundary conditions. 17

3.3 Example initial and final states for the incompressible flow past a cylinder dataset, shown on the irregular geometry together with the underlying point cloud. While the model is tasked with predicting the v_x and v_y components of the velocity field, these plots display the speed $\|\mathbf{v}\|$. 18

3.4 Example temperature field from the HTGR column dataset for a fixed boundary temperature and selected axial layer, shown together with the irregular spatial mesh used for sampling. 19

4.1 A plot of the MSE loss as a function of resolution for the four models trained on the Navier-Stokes dataset. The training resolution is indicated by the vertical dashed line. 24

4.2 A plot of the MSE loss as a function of resolution for the four models trained on the Darcy flow dataset. The training resolution is indicated by the vertical dashed line. 25

4.3 A plot of the MSE loss for four FNOs trained on the Navier-Stokes dataset. FNO is the standard FNO; the other models have been modified to include resampling onto a fixed internal grid and/or use the UND block used by the CNO. The training resolution is indicated by the vertical dashed line. 27

5.1 Representative geo-FNO prediction on the cylinder flow dataset. The panels show the initial velocity field, the ground-truth final velocity field, the model prediction, and the pointwise difference between prediction and ground truth. . 29

5.2 Representative geo-FNO prediction on the HTGR column temperature dataset. The panels show the ground-truth temperature field, the model prediction, and the difference between prediction and ground truth on the irregular spatial mesh. 30

B.1 Fourier spectra of an example initial and final vorticity field from the generated Navier-Stokes dataset. The initial spectrum reflects the truncated low-frequency initialization with Gaussian weighting, while the final spectrum shows the redistribution of spectral energy over time. 39

Chapter 1

Introduction

PDEs are very important in many scientific and engineering domains etc etc etc classical solvers expensive learning-based surrogates can perform similarly but produce answers in a fraction of the time standard machine learning approaches are vector to vector, we want function to function define discretization invariance and how it relates to neural operators/function to function architectures include research questions and motivation for the work, and a brief summary of the contributions of the thesis. The introduction should provide a clear overview of the problem being addressed, the significance of the research, and how the thesis is structured to address the research questions.

1.1 Research Question(s)

The research question investigated in this thesis is . . .

Main research question:

- To what extent can neural operator methods provide discretization-robust surrogate models for PDE-based data on both uniform grids and non-uniform point clouds?

Sub research questions:

- To what extent do FNO and CNO generalize across grid resolutions on uniform-grid PDE benchmarks?
- How effectively can Geo-FNO extend Fourier neural operators to non-uniform point-cloud data?
- What limitations are observed when applying Geo-FNO to reactor-relevant non-uniform data?

Chapter 2

Background & Theory

This chapter introduces the theoretical background needed for the remainder of the thesis. It first places operator learning in context with more classical learning setups and then discusses the neural operator architectures that are central to this work.

SOMEWHERE IN HERE: explain bandlimits/nyquist frequency and how this relates to the resolution of the data and the model's ability to learn the underlying operator. This is a key concept that motivates the CNO architecture and its stability across resolutions, as well as the challenges faced by FNO when dealing with high-frequency information that may not be well-resolved in the training data.

2.1 From standard machine learning to Operator Learning

This section introduces operator learning as a mapping between functions, rather than the more familiar vector-to-vector setting. It should explain why this perspective is useful for PDE-governed physical systems and introduce the CDE concept that motivates resolution-aware learning.

Introduce the continuous-domain view and connect it to the notion of discretization invariance used throughout the thesis.

2.1.1 Discretization Invariance

2.2 Fourier Neural Operator

This section should explain the FNO architecture and the spectral convolution mechanism on which it is based.

2.2.1 Architecture

Describe the lifting layer, Fourier layers, spectral convolution, non-linear transformations, and projection back to the target field.

2.2.2 Strengths

Discuss the global receptive field, computational efficiency through FFT-based convolutions, and the suitability of FNO for smooth solution operators.

2.2.3 Weaknesses

Discuss spectral truncation, sensitivity to resolution changes, and the reliance on regular grids.

2.3 Convolutional Neural Operator

This section should explain the CNO architecture and the inductive biases that differentiate it from FNO.

2.3.1 Architecture

Describe the encoder-decoder style structure, convolutional blocks, and the way multi-scale processing is used.

2.3.2 Bandlimited assumption

Explain the bandlimited assumption and why it can stabilize learning and support more consistent behavior across discretizations. Discuss how this means explicitly discarding high-frequency information, where the FNO keeps it (but does not know what to do with it).

2.3.3 Local versus global inductive bias

Compare the locality of convolutional operators with the global spectral bias of FNO.

2.4 Mesh-free Operator Learning and geo-FNO

This section motivates operator learning on irregular domains and introduces geo-FNO as the architecture used for this purpose.

2.4.1 Motivation for irregular domains

Explain why many scientific and engineering problems do not naturally live on uniform Cartesian grids and why this matters for neural operators.

2.4.2 geo-FNO

Describe the geo-FNO approach and how it extends Fourier-based operator learning to non-uniform meshes.

explain the architecture itself and that it is a generalization of FNO that collapses back to FNO when the input points are on a regular grid. Also mention that you can just interpolate the input data to a regular grid and use FNO, but this is not ideal because it can introduce interpolation errors and does not leverage the potential advantages of learning directly on the original point cloud. cite the geo fno paper that tested this and found lower loss when learning directly on the point cloud with geo-FNO compared to interpolating to a regular grid and using FNO.

2.5 Discretization invariance on mesh-free grids

More philosophical discussion of what discretization invariance means when we no longer have a regular grid and how this relates to the notion of a continuous-domain operator.

something like:

In this thesis, discretization invariance is interpreted as the ability of a model trained on data at one uniform resolution to make accurate predictions on the same underlying problem at different uniform resolutions. This notion does not transfer directly to non-uniform point clouds, where there is no single grid resolution. For non-uniform data, the relevant question is instead whether the sampled points sufficiently resolve the solution features needed for prediction.

talk about this more in method/results chapter when we discuss loss as function of resolution.

FINAL VERSION:

2.6 Operator Learning

Operator learning refers to the use of neural networks to approximate mappings between function spaces, rather than between finite-dimensional vectors as is done for classical neural network approaches [8]. This setting arises naturally in scientific computing, where many problems can be formulated in terms of operators acting on functions.

A particularly important example is the solution operator of a parametric partial differential equation (PDE). Consider a PDE of the form:

$$\mathcal{N}(a,u)(x) = f(x), \quad x \in \Omega, \qquad u(x) = 0, \quad x \in \partial\Omega, \tag{2.1}$$

where $\Omega \subset \mathbb{R}^d$ is a bounded domain, $\mathcal{N}$ is a differential operator, a is an input function, f is an external term like a forcing term, and u is the solution.

The corresponding solution operator is defined as:

$$\mathcal{G}(a) = u, \tag{2.2}$$

which maps an input function a to the solution u. The goal of operator learning is to approximate this mapping directly using surrogate modeling.

Let $\mathcal{A}$ and $\mathcal{U}$ denote appropriate function spaces for the input and output, respectively. Given a dataset of input-output pairs:

$$\{(a_j, u_j)\}_{j=1}^{N_{\text{data}}}, \quad u_j = \mathcal{G}(a_j), \tag{2.3}$$

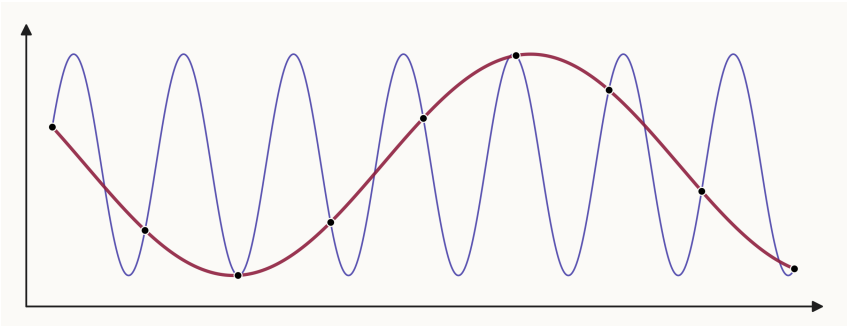

Figure 2.1: an illustration of aliasing. The blue curve represents a high-frequency signal, while the red curve is a lower-frequency function that passes through the same sampled points (black dots). Since both functions agree at the sampling locations, the discrete representation cannot distinguish between them.

where the inputs a_j are sampled from a distribution μ over $\mathcal{A}$, the objective is to learn a parametric approximation $\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}$ by minimizing the expected error

$$\mathbb{E}_{a\sim\mu}\left[\|\mathcal{G}(a) - \mathcal{G}_\theta(a)\|^2_{\mathcal{U}}\right], \tag{2.4}$$

which in practice is approximated by the empirical loss over the dataset.

A key distinction from standard learning problems is that the objects of interest are functions rather than finite-dimensional vectors, even though in practice they are only observed through discrete samples. Consequently, the learning problem is formulated at the level of a continuous operator, while training is carried out on its discretized representations. This makes it essential to understand what information about a function is preserved under discretization.

2.6.1 Bandlimitedness and Continuous–Discrete Equivalence

A central question in operator learning is what information about a continuous function is preserved when it is represented through discrete samples. Sampling theory provides a framework for answering this question, by characterizing when a function can be uniquely reconstructed from its values on a discrete grid.

Let $f \in L^2(\mathbb{R})$ be a square-integrable function with Fourier transform $\hat{f}$. The function is said to be *bandlimited* with bandlimit $\Omega > 0$ if its Fourier transform is supported only on a finite interval:

$$\operatorname{supp}(\hat{f}) \subset [-\Omega, \Omega]. \tag{2.5}$$

The Whittaker-Shannon sampling theorem states that a bandlimited function can be exactly reconstructed from its values on a uniform grid, provided the sampling rate is sufficiently high [11]. Specifically, if the function is sampled at points $\{f(nT)\}_{n\in\mathbb{Z}}$, then exact reconstruction is possible if

$$\frac{1}{T} \geq 2\Omega, \tag{2.6}$$

where $1/T$ is the sampling frequency and 2Ω is called the Nyquist rate.

Under this condition, the function can be reconstructed via the interpolation formula

$$f(x) = \sum_{n\in\mathbb{Z}} f(nT)\,\mathrm{sinc}\left(\frac{x-nT}{T}\right), \tag{2.7}$$

where $\mathrm{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$.

This result establishes a form of continuous-discrete equivalence: if a function is bandlimited and sampled above the Nyquist rate, then its discrete samples contain all the information required to reconstruct the original function. In this case, the discretization is effectively lossless.

However, if the sampling rate is below the Nyquist rate, this equivalence breaks down. In this case, different continuous functions can produce exactly the same set of discrete samples, meaning that the original function can no longer be uniquely reconstructed from the sampled data.

This phenomenon is known as aliasing. Intuitively, aliasing occurs when oscillations in the function are too rapid relative to the spacing between sample points. If the grid is too coarse, the samples cannot resolve the true high-frequency behavior, and the function appears to vary more slowly than it actually does.

Figure 2.1 illustrates this effect. The blue curve represents a high-frequency signal, while the red curve is a much lower-frequency function passing through the same sampled points. Since both functions agree exactly at the sampling locations, the discrete representation cannot distinguish between them. From the perspective of the sampled data, the high-frequency signal is therefore indistinguishable from its lower-frequency alias.

From a Fourier perspective, a sampled grid can only represent frequencies up to the Nyquist limit

$$\omega_N = \frac{\pi}{T}, \tag{2.8}$$

where T is the spacing between sample points. Frequencies satisfying

$$|\omega| \leq \omega_N \tag{2.9}$$

can be represented uniquely, while frequencies above this limit cannot.

Consider a sinusoidal component

$$f(x) = \sin(\omega x). \tag{2.10}$$

Sampling this function at points $x_n = nT$ gives

$$f(nT) = \sin(\omega nT). \tag{2.11}$$

If $\omega > \omega_N$, there exists a lower frequency $\omega_{\text{alias}} \leq \omega_N$ such that

$$\sin(\omega nT) = \sin(\omega_{\text{alias}} nT) \tag{2.12}$$

for all sample points n. A common expression for this aliased frequency is

$$\omega_{\text{alias}} = |\omega - 2k\omega_N|, \tag{2.13}$$

for some integer k chosen so that

$$0 \leq \omega_{\text{alias}} \leq \omega_N. \tag{2.14}$$

This shows that frequencies above the Nyquist limit do not simply disappear. Instead, they are systematically and deterministically mapped onto lower frequencies that the grid can represent. Aliasing is therefore not random noise, but a predictable distortion of the frequency content.

The consequence is that information above the Nyquist limit is irreversibly lost. Once aliasing has occurred, the sampled data contains a distorted representation of the original function, where unresolved high-frequency behavior is interpreted as physically different low-frequency structure. No learning algorithm can reconstruct frequency content that is absent from the discretized data, regardless of model complexity or architecture [1].

2.6.2 Discretization Invariance

The discussion in Section 2.6.1 shows that learning from discretized data is fundamentally constrained by the information preserved under sampling. Even if a model is intended to approximate a continuous operator, it is trained and evaluated on finite representations of functions. This raises the question of whether the learned model depends only on the underlying function, or also on the particular discretization used to represent it.

This property is commonly referred to as discretization invariance. Informally, a discretization-invariant model should produce consistent predictions when the same underlying function is represented on different grids, provided those discretizations sufficiently resolve the relevant features of the problem.

In practice, discretization invariance is most naturally interpreted for functions represented on uniform Cartesian grids, where resolution can be clearly defined through grid spacing. In this setting, it refers to the ability of a model trained at one resolution to make accurate predictions at other resolutions of the same underlying problem.

A widely used characterization of neural operators is given by Kovachki et al. [8], who state that a model should satisfy three key properties:

1. **Discretization flexibility:** the model can accept inputs defined on arbitrary discretizations of the domain.

2. **Pointwise evaluation:** the model can be evaluated at arbitrary output locations.

3. **Continuum consistency:** as the discretization is refined, the learned operator converges to the underlying continuous operator being approximated.

These properties formalize the idea that a neural operator should approximate a mapping between functions, rather than a mapping between specific finite-dimensional representations. In particular, the learned operator should depend on the underlying continuous object and not on the resolution, ordering, or structure of the grid used to represent it.

Since training data is always discretized, a model is never directly exposed to the continuous operator itself, but only to finite samples of its input and output functions. If refining

the discretization changes the behavior of the model rather than improving the approximation of the same underlying mapping, the model becomes tied to the representation used during training rather than to the operator itself. As a result, performance may degrade significantly when evaluated on different resolutions or different discretizations of the same problem. This phenomenon is commonly referred to as *discretization mismatch* [**?**].

Discretization invariance is therefore a central criterion for evaluating neural operator architectures. In this thesis, it is used both as a theoretical concept and as an empirical measure for comparing models across changes in resolution.

2.7 Fourier neural operator

use the section already written but not currently accessible (mr gpt). to this section: add something about how global mixing/keeping global information is useful for many physical systems (cite source)

2.8 Convolutional neural operator

make it in the style/tone/length/structure of the FNO section. prompt:

next is the convolutional neural operator. the CNO can be seen as an extension of the U-net neural network that have proven to be effective in fields like image processing (cite source).

insert figure of the CNO architecture

we start by introducing the U-net. the U-net is a convolutional neural network, meaning it uses a kernel to apply convolutions. when interpreting this through the lense of physical systems, this corresponds to local mixing, which is beneficial for many physical systems exhibiting strong locality like X and Y (cite source). (maybe mention that this contrasts with the global bias of the FNO? this somewhat breaks the flow and does not feel like it fits in the CNO introduction section. I think maybe this is best saved for the discussion/comparison section) One limitation of convolutional neural networks when applied to physical systems is that the physical meaning of a certain kernel size depends on the resolution, as it will look at different spatial scales (include a figure here? explain in more detail?). CNOs borrow from U-nets and address this issue by using a multi-scale architecture, where the convolutional kernels are applied at multiple resolutions; see figure 2.8. The input is first fed through a lifting layer acting on the feature space (read thoughts at the end about the structure here). the input then passes through a convolutional block, consisting of a convolutional kernel application followed by a nonlinearity. as the representation descends on the encoder side (rephrase/explain this more), the spatial resolution is decreased using interpolation-based downsampling while the feature space ”resolution” is increased. the input is again passed through a convolutional block, but as the kernel size is the same across all blocks, a different spatial scale is captured. on the decoder side, the coarse resolution is upsampled before another convolutional block is applied. to access higher frequency content that was lost to the downsampling, skip connections are in place so that the network can ”see” what the input

looked like on the same resolution on the encoder side. (maybe mention projection before final prediction is made, not sure if its necessary)

While the U-net structure succeeds at capturing multiple physical scales, it is not a discretization invariant architecture. The CNO makes several modifications to the U-net structure aimed at making the U-net discretization invariant/robust. First and most important is resampling onto a fixed internal grid/resolution. by doing this, the CNO always treats the same resolution (connect to bandlimitedness; we are implicitly assuming that the chosen resolution/the bandlimit it imposes captures sufficient detail to let the model learn the solution operator). when the resampling is downsampling, higher frequency content is explicitly truncated, which helps to stabilize the model. even if the CNO could access higher frequncy content at inference, if it has not seen it during training, it would not know what to do with it. explicitly removing the higher frequency content helps satabilize the model performance when testing on higher resolutions. at lower resolutions, upsampling should be equivalent to zero padding in fourier space (maybe unnecessary reference to fourier space) and should introduce minimal errors/discrepancies. regardless, interpolation errors are not unheard of. (decide whether to include an interpolation error section; could show the fourier power spectrum of navier stokes before and after resampling round trip, which shows that the real frequencies are retained, but higher frequency, unphysical/spurious content is generated by the round trip). the next modification of the CNO compared to the U-net is that the simple pointwise nonlinearity is superceded by a upsample-nonlinearity-downsample (UND) block. this change is made as the pointwise nonlinearity alone is not discretization invariant. The application of the nonlinearity creates higher frequency content. consider ... (mathematical proof of this fact). when the generated frequencies have frequencies above nyquist for a given resolution, they will fold down and contaminate the resolvable modes. As outlined in section 2.6.1, the aliasing pattern is deterministic and dependent on the resolution, which violates discretization invariance (this doesn't make any sense. for the CNO, the UND block is applied only after the resampling, so the aliasing pattern is always the same. why exactly does the UND block lead to a more discretization invariant model? why is it included, what is it intended to do, and why does it help?). to solve this, the CNO uses the aforementioned UND block. before the nonlinearity is applied, the data/input is first upsampled to a higher resolution, which is equivalent to zero padding in fourier space. when the nonlinearity is then applied, the higher frequency content has space to exist without folding into lower frequencies. after the nonlinearity is applied, the data is downsampled back to the original resolution using a X filter (I do not remember the name, but it is not a true low pass filter. it is an approximation. this is important to cover in some depth, as this is why the CNO without the UND block outperforms the standard CNO at high resolutions of navier stokes, so i need to be able to refer back to it here when i cover it in discussion), which is equivalent to truncating the higher frequency content. by doing this, the CNO can safely apply nonlinearities without generating spurious low-frequency content that would violate discretization invariance. There are some other minor tweaks, but they are not theoretically important. just make sure to not make it sound like resampling onto a fixed internal grid + UND are the only changes, they are just the main ones

for strengths, the CNO is a robust model. the initial resampling ensures stable performance across resolutions,given that the training resolution is sufficient to capture the

relevant features of the problem. convolutions are useful for many physical problems that exhibit strong local bias like X and Y (cite source) (I see now that this is very similar to something i wrote earlier in the CNO section. I think it makes sense to mention at depth here; remove or shorten it where i wrote it before). the multi-scale architecture allows it to capture a wide range of physical scales, which is important for many physical problems. it is especially useful for problems like Navier stokes that see energy transfer across length scales (cite source). the UND block also ensures that it is a true neural operator, as it can apply nonlinearities without generating spurious low-frequency content that would violate discretization invariance [1]. others?

for weaknesses, the CNO is quite computationally expensive due to the use of convolutional layers and the multi-scale architecture. the changes made compared to a U-net also add to the computational cost. it also has more hyperparameters to tune, which can make training more difficult (is this true?). the resampling step can introduce interpolation errors, which can degrade performance if not done carefully. Others?

one problem i see with this structure is that i explain a U-net in some depth, but i do not make it clear whether this is about the u-net or the CNO. i later explain in which way the CNO builds on and ”improves” the U-net, and only then do i explain that the CNO first resamples the input onto a fixed internal grid. this was not mentioned in my explanation of the u-net structure, which could confuse the reader. the only reason i dont make it crystal clear that I am talking about u-nets when i first explain this is that i am referring to a figure that shows the CNO structure. it is close enough to the U-net that keeping both seems unnecessary.

2.9 Mesh-free Operator Learning and geo-FNO

short intro section? or straight to irregular domains?

2.9.1 Motivation for irregular domains

While successful, the operators introduced in section **??** and **??** run into the issue of requiring the input data to be structured on a uniform, rectangular, Cartesian grid. This is often a very inefficient way of storing the data. a dataset with high variance in one area and low variance in another can much more effectively capture the dataset with varying sampling rates (cite source). other datasets may simply be on a geometry that does not lend itself to a Cartesian representation (cite source). While one could always interpolate a dataset of an arbitrary structure onto a Cartesian grid and feed that to uniform grid neural operators, this has been shown to incur signicant interpolation errors, hampering the performance of these models significantly (cite source; i think there was a good one in the introduction of the geo fno paper). This motivates extending neural operators to non-uniform meshes/point clouds. other things to bring up here?

2.9.2 geo-FNO

The geo-FNO is one of the most significant non-uniform neural operators. The architecture directly builds upon the original FNO, effectively being a generalization of it (cite source). The geo-fno, short for geometry-aware FNO, consists of a coordinate transform in the form of a light MLP in front of an FNO as described in section **??**. the coordinate transform, referred to as IPHI (use inverse of greek letter phi), is trained jointly with the FNO and thus learns a transform that minimized the loss. the grid following the transformation is referred to as the computational grid. this non uniform grid is the first input to the FNO, with the FFT replaced with a non uniform DFT. after the first Fourier layer, when the inverse fourier transform is taken, we return to physical space on a uniform grid, which is used throughout the FNO until the final output. this compromise delivers better performance than the interpolation onto a uniform grid outlined in section **??**, while maintaining the efficiency of fast fourier transforms within the FNO.

The geo-FNO has the advantage that it can be evaluated at arbitrary positions. This is because the final FNO layer has a stage where it has the fourier coefficients of its solution prediction, which is defined everywhere in physical space. The geo-FNO allows for a list of points to be passed, and prediction can be made on said points despite these points not being seen during training. The geo-FNO therefore satisfies two of the points outlined in section **??**, as it can take input and output at arbitrary positions.

iphi is a coordinate transform that takes the form: iphi(x, a) = x + f(x, a) where x is the set of input points and a is an optional vector of geometry parameters. f(x,a) is initialized as identity, stabilizing training. iphi is trained jointly with the FNO, meaning that the coordinate transform learns to minimize the loss while adapting to the geometry of the input data. This could mean mapping to a distribution where energy is shifted away from the higher modes that the FNO cannot resolve and into lower modes that are utilized. note that iphi does not map onto a discrete grid; this is a continuous to continuous mapping. the discrete fourier transform has its problems, as the resulting Fourier basis is in general not orthogonal (include proof of this; show why the basis is orthogonal for a uniform grid, and explain which step does not hold for non uniform grids). this could cause issues because ... (i am actually not sure what the consequences could be; help me with this). the vector a lets the model access information about geometric parameters that could differ between samples. an example of these is the aerospace airfoil thingy i need to find a source for, where something like 20-30 different parameters together span a continuous space. by feeding iphi these parameters, the mapping can be tailored to the exact geometric setup, letting the geo-FNO make a wider range of predictions more efficiently. (somewhere in the iphi section: include a couple of lines + source showing that such a coordinate transform can be effective. ex. chebyshev basis from original geo fno paper)

are there other things to mention about the geo-FNO? it is really just a coordinate transform wrapper around an FNO already described in the FNO section. the FNO itself is unchanged except for the initial fourier transform and final inverse transform being replaced by non-uniform approximations. let me know if i am missing something or if this is enough

2.10 Discretization invariance on mesh-free grids

More philosophical discussion of what discretization invariance means when we no longer have a regular grid and how this relates to the notion of a continuous-domain operator.

In this thesis, discretization invariance is interpreted as the ability of a model trained on data at one uniform resolution to make accurate predictions on the same underlying problem at different uniform resolutions. This notion does not transfer directly to non-uniform point clouds, where there is no single grid resolution. For non-uniform data, the relevant question is instead whether the sampled points sufficiently resolve the solution features needed for prediction. This is more a question of where those points are instead of simply how many there are.

help me with this section. what i really want to say is that discretization invariance is an easy concept for uniform grids, as an increase in resolution is well defined. on non uniform grids/point clouds, this is more complicated. the sheer number of points is not nearly as important as their distribution. i want to explain that because of this increase in complexity, no resolution experiments were conducted for non uniform grids. i want to have more of an explanation than that, however, and explain how my results and the fact that the models produces proper predictions is indicative of ... the section above i think is a good starting point

talk about this more in method/results chapter when we discuss loss as function of resolution.

Chapter 3

Experimental Setup

This chapter describes the datasets and training procedures used throughout the thesis. It also clarifies how models with different output-resolution behavior are compared in the discretization transfer experiments.

3.1 Datasets

The experiments in this thesis use both uniform-grid and non-uniform datasets. For each dataset, the goal of this section is to describe the underlying physical problem, the mapping learned by the model, the data source, and the aspects of the dataset that are relevant for interpreting the results. Dataset-specific implementation details are included only when they affect the conclusions drawn later in the thesis. In particular, the full generation procedure for the self-generated Navier-Stokes dataset is deferred to Appendix B. For all datasets, 80% of the samples were used for training and 20% for testing, unless otherwise noted.

3.1.1 Uniform-grid datasets

The discretization invariance experiments on regular grids use two datasets: a self-generated Navier-Stokes dataset and a Darcy flow dataset obtained from prior work.

Navier-Stokes

For the regular-grid experiments, we use a self-generated two-dimensional Navier-Stokes dataset on the unit square. The Navier-Stokes equations are among the fundamental equations of fluid dynamics and arise in many areas of physics and engineering, including atmospheric and oceanic flows, aerodynamics, and industrial transport processes. The dataset is formulated and solved in vorticity form, with governing equation:

$$\partial_t \omega + u \cdot \nabla \omega = \nu \Delta \omega + f, \tag{3.1}$$

where ω denotes the vorticity field, u the incompressible velocity field, ν the viscosity, and f an external forcing term. The learning task considered in this thesis is not a pressure or velocity prediction problem, but an operator mapping from an initial vorticity field to

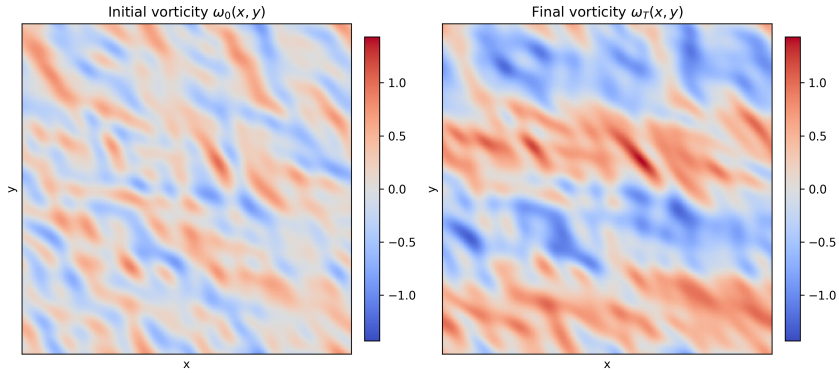

Figure 3.1: Initial and final vorticity fields for a sample from the Navier-Stokes dataset.

a later vorticity field. The input to the model is the initial vorticity field $\omega(\cdot, 0)$, and the output is the vorticity field at a later time $\omega(\cdot, T)$. An example input-output pair is shown in Figure 3.1.

The dataset consists of 1000 samples generated at a resolution of 256×256. Between samples, the viscosity and forcing remain fixed, while the initial vorticity field is varied.

Further details on the data-generation procedure are provided in Appendix B.

ADD SOMETHING MORE HERE: mention that we saw some strange behavior for the geo-FNO, namely the central artifact/seam in predictions for all datasets. As part of the troubleshooting process, we generated different versions of the Navier-stokes dataset with different sampling patterns. In addition to the uniform grid seen above, a central-clustered, left-clustered and near uniform grid were produced. This dataset is well suited for this purpose as the initial condition is generated in Fourier space, which is continuously defined in physical space. This lets us generate the same underlying physical dataset on different spatial discretizations, which is not possible for the other datasets. Refer to whatever section you end up explaining the seam and how it was fixed in. Decide if this was actually useful for troubleshooting purposes or not and whether to include it, but if you do, it should be mentioned here.

Darcy flow

Also used for the uniform grid experiments is the Darcy flow dataset introduced by Li et al.[9]. In this benchmark, the task is to learn the solution operator of a steady-state two-dimensional Darcy flow problem on the unit square, governed by the following PDE:

$$-\nabla \cdot (a(x)\nabla u(x)) = f(x), \qquad x \in (0,1)^2, \tag{3.2}$$

$$u(x) = 0, \qquad x \in \partial(0,1)^2, \tag{3.3}$$

The input is a spatially varying diffusion coefficient field $a(x)$, and the output is the corresponding solution field $u(x)$ under a prescribed forcing term and Dirichlet boundary

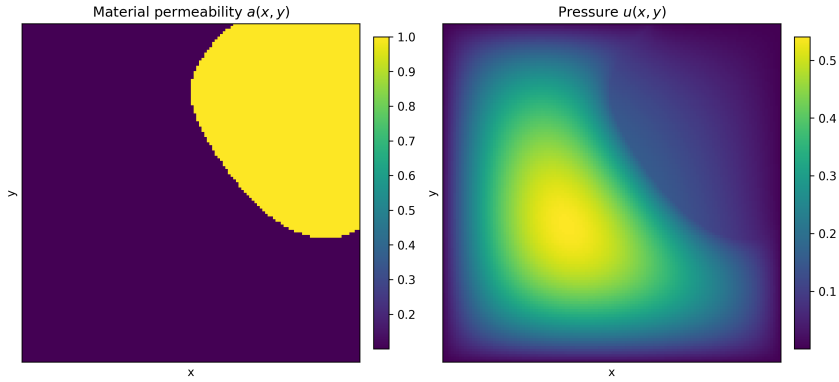

Figure 3.2: Example of an input diffusion coefficient field $a(x,y)$ and the resulting pressure field $u(x,y)$ with Dirichlet boundary conditions.

conditions. An example input-output mapping can be seen in figure 3.2. Between samples, $f(x)$ is kept constant while the $a(x)$ is varied. This PDE has many applications within physics and engineering, including drainage and irrigation in agricultural soils and fluid migration around nuclear waste repositories [10]. For this thesis, the Darcy flow dataset serves as a regular-grid benchmark for evaluating the performance of FNO and CNO across discretizations. This dataset was selected because it provides a comparatively smooth benchmark, complementing the more irregular and turbulent behavior of the Navier-Stokes dataset discussed in Section 3.1.1. Using both datasets makes it possible to evaluate model performance under qualitatively different solution regimes. The dataset contains 10000 samples at a resolution of 128×128.

3.1.2 Non-uniform datasets

The experiments on irregular domains primarily use two datasets: an incompressible Euler dataset modeling flow past a cylinder and a nuclear engineering dataset consisting of temperature profiles for a single HTGR column.

Elasticity

The elasticity dataset is included because it is one of the canonical benchmarks used to evaluate geo-FNO on non-uniform meshes. It provides a direct connection to the original geo-FNO setting and therefore helps assess whether the implementation used in this thesis reproduces the expected behavior on an established test case.

In the dataset description, it is enough to explain the physical problem at a high level, describe the input and output quantities, and note that the data is defined on an irregular geometry rather than a uniform Cartesian grid. The main text should also cite the original source and report the experimental resolution or mesh representation at the level needed to interpret the results.

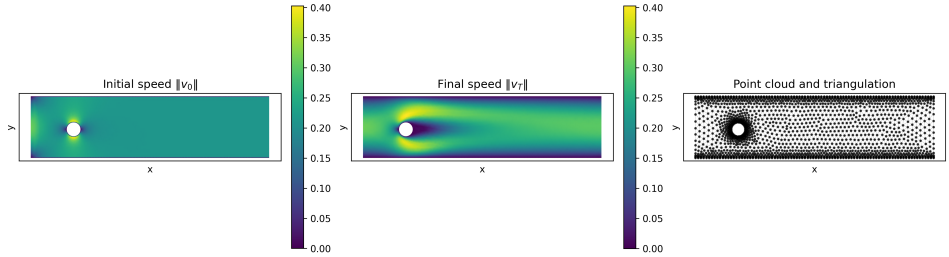

Figure 3.3: Example initial and final states for the incompressible flow past a cylinder dataset, shown on the irregular geometry together with the underlying point cloud. While the model is tasked with predicting the v_x and v_y components of the velocity field, these plots display the speed $\|\mathbf{v}\|$.

Should I include this at all? maybe for the troubleshooting section for finding the seam. Maybe add another subsection after non uniform grids called troubleshooting grids where I just add elasticity and non uniform NS

Incompressible Navier-Stokes flow past a cylinder

A second non-uniform benchmark is an incompressible Navier-Stokes dataset describing fluid flow past a cylinder. This dataset was used in prior work on operator learning for PDEs and is included here because it provides a fluid-dynamics test case on a geometry that is not naturally aligned with a regular grid [7, 12].

The dataset describes two-dimensional incompressible flow with velocity $\mathbf{v}$ and pressure p, governed by:

$$\nabla \cdot \mathbf{v} = 0, \tag{3.4}$$

$$\rho_0 \left(\partial_t \mathbf{v} + \mathbf{v} \cdot \nabla \mathbf{v}\right) + \nabla p = \mu \nabla^2 \mathbf{v}, \tag{3.5}$$

where ρ_0 is the constant density and μ the viscosity. The domain consists of a channel containing a cylindrical obstacle, with the cylinder position and diameter varying between samples.

The prediction task is to learn a mapping from the velocity field at the initial time to the velocity field at the final time, while accounting for the fact that changes in cylinder size and location lead to different corresponding flow patterns. In the experiments of this thesis, the model is trained to predict only the horizontal and vertical velocity components; pressure is not included in the target.

This dataset is particularly relevant for geo-FNO because it is defined on a non-uniform spatial discretization rather than a regular Cartesian grid. An example initial-final pair is shown in Figure 3.1.2. The dataset consists of 1000 samples each containing between 1700 and 2100 points, depending on the geometry of the sample. 200 of these samples were designated for testing, while the remaining 800 were used for training.

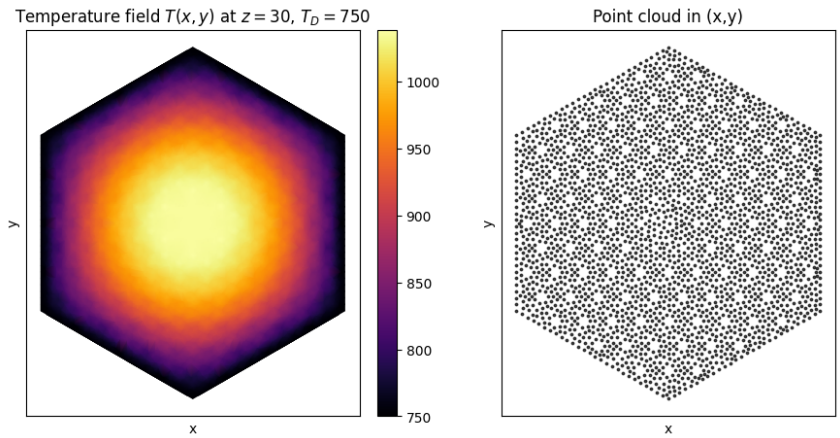

Figure 3.4: Example temperature field from the HTGR column dataset for a fixed boundary temperature and selected axial layer, shown together with the irregular spatial mesh used for sampling.

HTGR column temperature dataset

The final non-uniform dataset consists of steady-state temperature profiles for a single column of a prismatic core high temperature gas-cooled reactor (HTGR). This dataset is included because it provides a first application-oriented test case for geo-FNO in a nuclear engineering setting. Unlike the public benchmark datasets considered earlier, the data was produced internally as part of ongoing work by a colleague.

Physically, the dataset describes the temperature distribution within a three-dimensional reactor column. For each case, a constant boundary temperature is imposed and a coupled neutronics-thermohydraulics calculation is performed until a steady state is reached. The transient evolution toward this steady state is not included in the dataset; only the final steady-state temperature fields are used. The use of a fixed boundary temperature is a simplifying assumption, since in a realistic reactor system the boundary temperature would itself change as the system evolves. This simplification was made to produce a controlled dataset within the time constraints of the project.

The spatial sampling locations lie on an irregular mesh that is shared across all samples. Along the axial direction, the column is quantized into 17 distinct z-levels. The imposed boundary temperature ranges from 300 °C to 1000 °C in steps of 1 °C, yielding 701 distinct thermal conditions. Since the prediction task is defined on individual horizontal slices rather than on the full three-dimensional column, this results in a total of $17 \times 701 = 11917$ two-dimensional samples.

In the prediction task considered in this thesis, the model receives the boundary temperature together with a specified z-level and predicts the temperature at all sampled (x, y) locations in the corresponding horizontal slice. The task can therefore be interpreted as learning a mapping from the boundary condition and axial position to the two-dimensional temperature profile within that layer. An example of such a slice, together with the under-

lying irregular mesh, is shown in Figure 3.4.

3.2 Training setup

All models are trained in a supervised setting on paired input-output samples from the datasets described above. For most datasets, the data is split into a training set and a held-out evaluation set. The training set is used to optimize the model parameters, while the held-out set is used to monitor performance during training and to report final model performance.

The hyperparameters differ between the uniform-grid and non-uniform-grid experiments, as well as between datasets. To keep the main text focused on the experimental design rather than implementation details, full hyperparameter settings, including learning rates, batch sizes, training epochs, and architecture parameters, are given in Appendix D.

3.3 Uniform-grid resolution transfer experiments

The uniform-grid experiments are designed to evaluate discretization invariance by testing how well a model trained at one spatial resolution generalizes to other resolutions. The Navier-Stokes and Darcy flow datasets described in 3.1.1 and 3.1.1 respectively are used for these experiments. For each dataset, the models are trained at a chosen training resolution and then evaluated both at the training resolution and at lower and higher resolutions. This makes it possible to compare performance at the training resolution with performance after transferring the same trained model to different spatial resolutions.

For the Navier-Stokes dataset, the models are trained at a resolution of 192×192 and evaluated on resolutions from 128×128 to 256×256 in steps of 16 grid points. For the Darcy flow dataset, the models are trained at a resolution of 96×96 and evaluated on resolutions from 64×64 to 128×128 in steps of 8 grid points.

Four model variants are compared for each dataset. The first two are FNO models retaining either 8 or 16 Fourier modes in each spatial direction, denoted FNO_k8 and FNO_k16. The other two are CNO variants. The first is the regular CNO architecture as outlined in section REFER TO THEORY SECTION HERE, referred to as CNO_standard. The second is a modified CNO in which the upsample-nonlinearity-downsample (UND) operation is replaced by a simple pointwise nonlinearity, referred to as CNO_no_UND. This operation is one of the architectural changes that distinguishes the CNO from a U-Net-like architecture and is intended to reduce aliasing effects introduced by nonlinearities. Comparing the standard and modified CNO therefore makes it possible to assess the effect of this component in the resolution-transfer setting.

FNO and CNO handle changes in resolution differently. FNO produces predictions at the same resolution as the input it is given. When an FNO is evaluated at a resolution $N \times N$, both the input and target field are resampled to $N \times N$, after which the FNO prediction can be compared directly to the target field on that grid. CNO, however, produces predictions on its internal resolution. Therefore, when CNO is evaluated on an input resolution different from its internal resolution, the CNO prediction is resampled onto the input resolution before the error is computed. The target field is resampled to the same resolution, so that both FNO

and CNO are evaluated against target fields on the same spatial grid. All resampling in these experiments is performed with bicubic interpolation.

In addition to reporting the loss at each tested resolution, a scalar measure is used to summarize how strongly the loss changes away from the training resolution. For a set of tested resolutions R with loss L_N at resolution N[1] and with training resolution N_{train}, this score is defined as:

$$D = \frac{1}{|R|} \sum_{N \in R} \left| \log(L_N) - \log(L_{N_{\text{train}}}) \right| . \tag{3.6}$$

Lower values of D indicate that the loss remains closer to the loss at the training resolution across the evaluated resolutions. This score is only a measure of consistency across resolutions, not of absolute model quality. The loss values themselves remain the primary measure of predictive performance, while D is used only as a quantitative measure of discretization robustness.

3.4 Non-uniform-grid geo-FNO evaluations

The non-uniform-grid evaluations use geo-FNO on the cylinder flow and HTGR column temperature datasets described in Section 3.1.2. These evaluations differ from the uniform-grid resolution transfer experiments in Section 3.3. Since the spatial samples are given as point clouds or irregular meshes rather than Cartesian grids, there is no single resolution parameter equivalent to the grid size N used in the uniform-grid experiments. The goal is therefore not to measure resolution transfer, but to evaluate whether geo-FNO can learn mappings on irregular spatial data.

Feature engineering was used in the non-uniform experiments to augment the raw model inputs with additional variables that make relevant structure in the data more explicit. In general, feature engineering refers to the transformation or augmentation of raw input variables into representations that are more informative for a machine learning model, and prior work has shown that such transformations can improve predictive performance [6].

TO BE EXPLAINED HERE: CONNECT CYLINDER FLOW TO THEORY ON GEO-FNO. EXPLAIN THAT ”CODE” HERE IS THE X AND Y COORDINATES OF THE CYLINDER’S CENTER AND ITS RADIUS. THIS IS SEPARATE FROM THE POINTWISE QUANTITIES OUTLINED IN THE FEATURE ENGINEERING APPENDIX

For the cylinder flow dataset, the model did not produce meaningful predictions when it was given only the velocity field on the irregular mesh. The coordinate transform MLP ϕ^{-1} was given the x and y coordinates of the cylinder’s center and its radius. In addition to the horizontal and vertical velocity components, each mesh point was also assigned a one-hot encoding of its node type. The node-type categories distinguish the upper, lower and cylinder boundary nodes, left boundary nodes, right boundary nodes, and interior nodes. This made the location of the obstacle more explicit in the pointwise input representation, allowing the model to identify which mesh points lie on or near the cylinder.

[1] All evaluated domains are square. The resolution is therefore reported by its one-dimensional grid size N, while the full discretized domain contains N^2 grid points.

For the HTGR column temperature dataset, the raw input consists only of the boundary temperature T and the axial level z. Since this is a low-dimensional representation of the input conditions, several alternative representations of T and z were tested. These input representations and their effect on model performance are discussed in Appendix C. The best performing representation was a trigonometric encoding of z and T and was used for the results reported in the main text.

For the HTGR column temperature dataset, a separate train-validation-test split is used. The split is constructed over individual two-dimensional slice samples, where each sample corresponds to one combination of boundary temperature and axial level. Unlike a random split, the validation and test samples are selected using fixed subsets of the (z, T) input space.

The non-uniform-grid results should therefore be interpreted as held-out-data evaluations of geo-FNO on irregular spatial representations, rather than as a direct analogue of the uniform-grid discretization-invariance study. Implementation issues encountered during development, including the seam artifact observed in early geo-FNO predictions, are discussed separately in Appendix E. The results in the main text are reported using the corrected implementation.

Chapter 4

Uniform grids - Results and Discussion

4.1 results

4.1.1 Navier-Stokes

IMPORTANT: mention again somewhere that the navier stokes dataset is synthetic. This is necessary to understand some of the results, like how FNO_k16 performs so much better than FNO_k8 for Navier-Stokes than it does for Darcy flow. It also explains why the the FNO_k16 with added resampling onto a fixed internal grid becomes a flat line. not only is this just a good way to improve the performance of the FNO, but the 16 modes covers the bandwidth of the dataset, thus the model performs as well as it does on any resolution greater than 32x32.

ALSO: SHOW SINGULAR PLOTS OF MODEL PREDICTIONS VS GROUND TRUTH + DIFFERENCE PLOT.

The four models outlined in Section 3.3 were evaluated on the Navier-Stokes dataset. Figure 4.1 shows the MSE loss as a function of evaluation resolution for each model, with the training resolution indicated by the vertical dashed line. Table 4.1 reports the discretization robustness score D, which measures how much the loss changes across resolutions relative to the loss at the training resolution.

Among the FNO models, FNO_k16 achieves the lowest loss at the training resolution,

Table 4.1: Discretization robustness score D for the Navier-Stokes uniform-grid experiment. Lower values indicate that the loss remains more consistent across evaluation resolutions relative to the loss at the training resolution.

Model	D
FNO_k8	0.5076
FNO_k16	3.4817
CNO_standard	0.5854
CNO_no_UND	0.5847

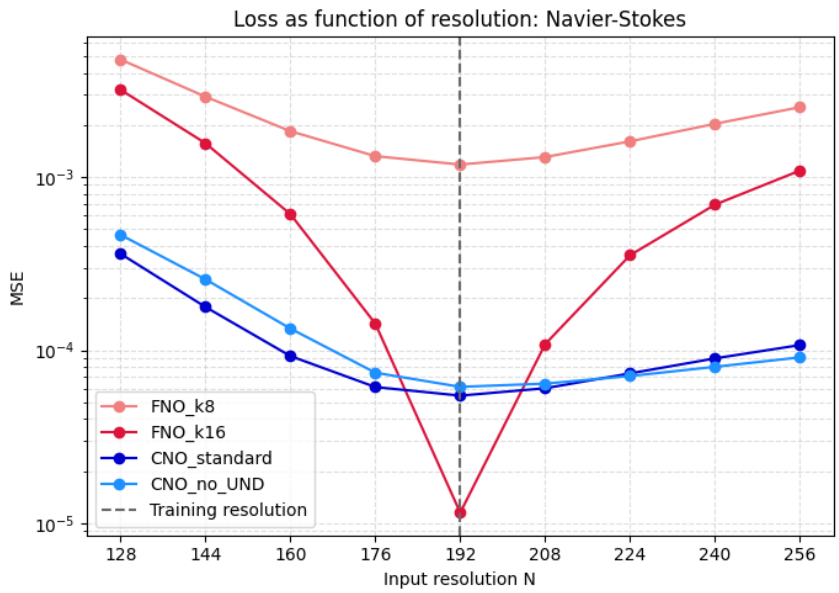

Figure 4.1: A plot of the MSE loss as a function of resolution for the four models trained on the Navier-Stokes dataset. The training resolution is indicated by the vertical dashed line.

but also has the largest value of D. Its performance therefore changes the most under resolution transfer. This contrast is especially pronounced at the lowest tested resolution, where the loss of FNO_k16 increases by more than two orders of magnitude compared with its loss at the training resolution. FNO_k8 has a higher loss at all tested resolution including the training resolution, but a substantially lower value of D, indicating a more consistent loss across the evaluated resolutions. For both FNO variants, the loss increases more rapidly when evaluating below the training resolution than when evaluating above it.

The two CNO variants show much flatter loss curves than FNO_k16 and have nearly identical values of D. Their training-resolution losses are also comparable. The standard CNO performs slightly better at the training resolution, at lower resolutions, and at $N = 208$. At the highest evaluated resolutions, CNO_no_UND gives slightly lower losses than CNO_standard. Overall, the Navier-Stokes results show a trade-off: FNO_k16 gives the best performance at the training resolution, while FNO_k8 and the CNO variants show more consistent behavior across resolutions.

4.1.2 Darcy flow

To test whether the trends observed on the Navier-Stokes dataset also appear on a qualitatively different PDE benchmark, the four models were evaluated on the Darcy flow dataset. The Darcy flow dataset was chosen as the solution space is smooth, making it a good point of comparison to the Navier-Stokes dataset. Figure 4.2 shows the MSE loss as a function of evaluation resolution for each model, with the training resolution indicated by the vertical dashed line. Table 4.2 reports the corresponding discretization robustness score D.

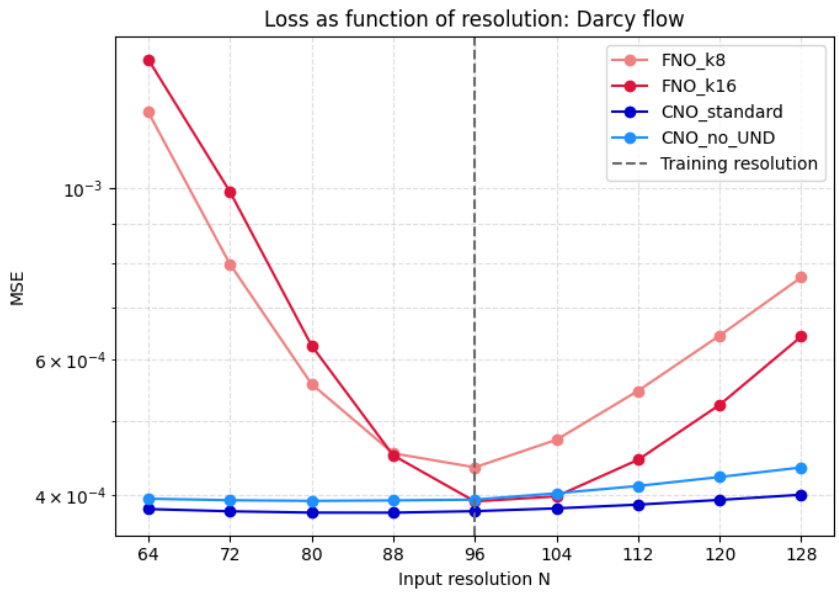

Figure 4.2: A plot of the MSE loss as a function of resolution for the four models trained on the Darcy flow dataset. The training resolution is indicated by the vertical dashed line.

Table 4.2: Discretization robustness score D for the Darcy flow uniform-grid experiment. Lower values indicate that the loss remains more consistent across evaluation resolutions relative to the loss at the training resolution.

Model	D
FNO_k8	0.3583
FNO_k16	0.4179
CNO_standard	0.0140
CNO_no_UND	0.0260

Compared with the Navier-Stokes experiment, the four models are closer in overall performance on the Darcy flow dataset. At the training resolution, CNO_standard achieves the lowest loss, followed by FNO_k16, CNO_no_UND, and FNO_k8. The two FNO variants show a similar pattern to that observed for Navier-Stokes; the loss is lowest near the training resolution and increases when the model is evaluated at other resolutions. This increase is stronger below the training resolution than above it. Notably, FNO_k8 outperforms FNO_k16 at resolutions below $N = 88$. This indicates that retaining more Fourier modes does not consistently improve performance under transfer to coarser grids.

The CNO variants show much flatter loss curves across the evaluated resolutions, with D values of order 10^{-2}. At lower resolutions, both CNO models maintain an almost constant loss, while at higher resolutions both losses increase slowly. In contrast to the Navier-Stokes experiment, CNO_standard outperforms CNO_no_UND at all evaluated resolutions. The loss of CNO_no_UND also increases more quickly than that of CNO_standard when

evaluating above the training resolution. Overall, the Darcy flow results show a clearer separation between the FNO and CNO variants in terms of discretization robustness; the FNO models are more sensitive to resolution transfer, while the CNO models remain nearly constant across the tested range.

4.2 Discussion/Interpretation/Analysis

This section should connect the observations back to the theoretical differences between the models and explain why we see the patterns we do in the results.

4.2.1 FNO

The FNOs perform poorly in terms of discretization invariance on both datasets. On Navier-Stokes, FNO$_k 8 has a comparable D - value to the CNOs, albeit with a significantly higher loss on all tested resolutions. This$

On the darcy flow dataset, we see that FNO$_k 8 performs better at lower resolutions. This is because, at lower resolutio$

the FNO's performance drops off faster at lower resolutions than at equally larger resolutions. (why? I do not remember.)

Mention that FNO on navier stokes performs artificially well. the dataset is generated in fourier space with 16 modes, so an FNO retaining 16 modes in each direction is naturally suited to this prediction task.

4.2.2 CNO

Discuss the role of locality, the bandlimited assumption, and the implicit multiscale structure in producing steadier behavior across resolutions.

As the CNO always resamples onto the same internal grid, the model is very stable. on darcy flow, the loss remains almost constant at lower resolutions; this is because the solution operator and solution space for this dataset is very smooth, and thus the tested lower resolutions still capture sufficient detail. at higher resolutions, we see that the loss on both datasets increases almost linearly. this is because the CNO always bandlimits the input. the rate at which the loss increases when testing at higher resolutions is therefore proportional to the rate at which the bandlimited approximation diverges from the "true" field.

We observe that the inclusion of the UND block improves performance for the CNO at all tested resolutions except high resolutions on the Navier-Stokes dataset specifically. This can be understood by considering what exactly the UND block does. As described in section **??**, the goal is to make the model more discretization invariant by not letting higher frequency content created by the nonlinearity fold into the retained modes. however, as mentioned in the theory section, the low pass filter applied during the downsampling back to the original resolution is not ideal. The utilized low pass filter effectively works by smoothing dataset, which works to erase high frequency content. While this works well to remove unresolved modes, it also affects and smooths the highest retained modes. This means that the highest retained modes are no longer available to the

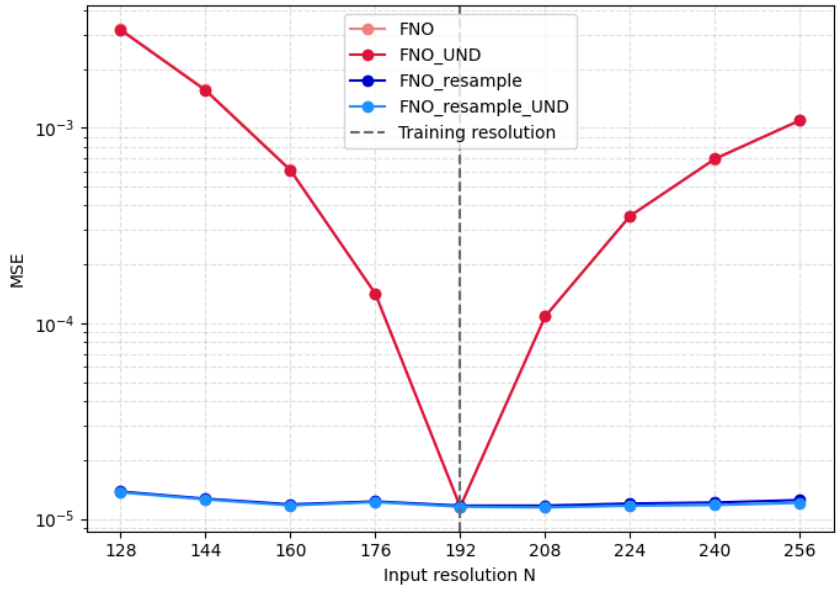

Figure 4.3: A plot of the MSE loss for four FNOs trained on the Navier-Stokes dataset. FNO is the standard FNO; the other models have been modified to include resampling onto a fixed internal grid and/or use the UND block used by the CNO. The training resolution is indicated by the vertical dashed line.

model to the same extent they would be without the smoothing. In other words, the inclusion of the UND block effectively applies an even stricter bandlimit to the input representation than the one enforced by the initial resampling. For the Darcy flow dataset, this has little consequence; the operator is heavily smoothing, so low frequency content dominates the solution space. Diminished access to the highest modes is therefore not detrimental. This is, however, not the case for the highly non-linear Navier-Stokes. The loss increase for CNO when tested at higher resolution is approximately linear on Navier-Stokes, and as already mentioned, the rate of loss increase is tied to the rate at which the training representation diverges from the higher bandwidth representations. as the CNO$_n o_U NDeffectivelyretainsaccesstomoremodes, itstandstoreasonthatitdivergesfromhigherbandwidthrep$

I fear I speak too much in frequencies here, when it's not clear why this would be the relevant lens through which to look at the CNO. is there a more physical explanation that works? should i have a disclaimer about this? or would a reader in this field be fine with this way of looking at this

4.2.3 improving the FNO

The FNO was modified to include parts of the CNO deemed responsible for its superior discretization invariance. These parts were found to be the UND block and the resampling onto a fixed grid. Four FNOs were examined; a standard FNO, an FNO with UND, an FNO with resampling, and an FNO with both UND and resampling. The models all retained

Table 4.3: Discretization robustness score D for the FNOs modified to improve discretization robustness on the Navier-Stokes dataset.

Model	D
FNO	3.4817
FNO_UND	3.4800
FNO_resample	0.0490
FNO_resample_UND	0.0456

16 modes in each direction and were trained and tested on the Navier-Stokes dataset. The loss under resolution change was then tested with the results seen in figure 4.3 and the corresponding D-values in table 4.3.

The UND block had a neglible, but positive, impact. The D-values are slightly lower for the FNOs with UND when compared to the equivalent FNO without. This shows aliasing is an issue, but that it is not the primary failure mechanism causing the poor discretization invariance. Much more notably, the FNOs with resampling show a dramatic improvement in discretization robustness. The D-values for the FNOs with resampling are an order of magnitude lower than those without. This is because it fixes the problem outlined in section **??**: the equivalence between mode index and physical meaning. By resampling onto a fixed internal grid, this equivalence is restored, even when testing the model at different resolutions. By implementing parts of the CNO architecture, the FNO's performance under resolution transfer is dramatically improved.

Chapter 5

Non-uniform meshes - Results and Discussion

5.1 Results

5.1.1 Cylinder flow

Geo-FNO was first evaluated on the cylinder flow dataset. Figure 5.1 shows a representative prediction on a held-out test sample.

The prediction captures the main structure of the final velocity field, including the global flow pattern around the cylinder and the downstream wake. The difference plot shows that the error is not distributed uniformly across the domain. Larger errors occur primarily in regions where the velocity field changes more sharply, especially near the wake and around the obstacle. Despite these local errors, the qualitative agreement between the prediction and the ground truth indicates that geo-FNO can learn the velocity-field mapping directly on the irregular point-cloud representation.

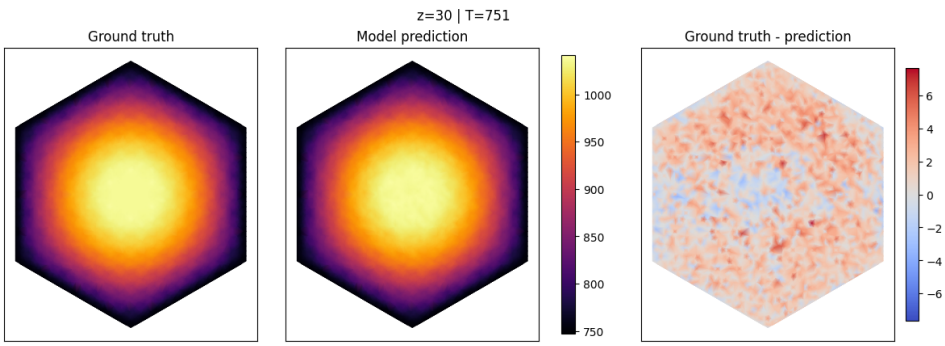

Figure 5.1: Representative geo-FNO prediction on the cylinder flow dataset. The panels show the initial velocity field, the ground-truth final velocity field, the model prediction, and the pointwise difference between prediction and ground truth.

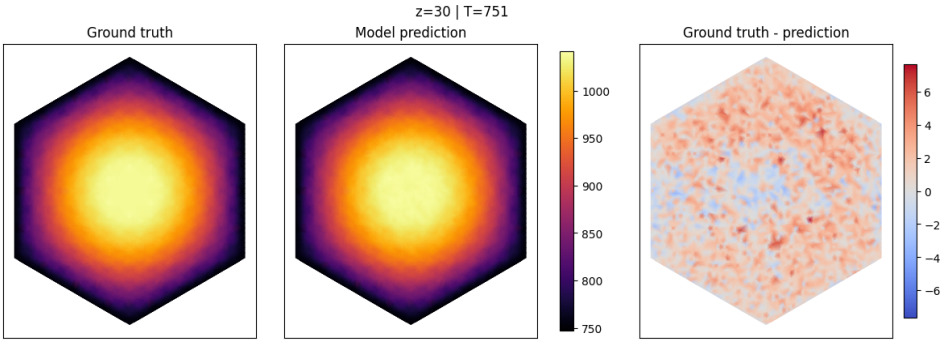

Figure 5.2: Representative geo-FNO prediction on the HTGR column temperature dataset. The panels show the ground-truth temperature field, the model prediction, and the difference between prediction and ground truth on the irregular spatial mesh.

5.1.2 HTGR column temperature dataset

Geo-FNO was evaluated on the HTGR column temperature dataset described in Section 3.1.2. Figure 5.2 shows a representative prediction on a held-out sample.

The predicted temperature field closely matches the ground truth. The dominant spatial temperature pattern is reproduced, and the difference plot shows small local deviations relative to the overall temperature scale.

5.2 Discussion/Interpretation/Analysis

This section should connect the non-uniform results back to the role of geo-FNO as an extension of FNO to irregular spatial data. It should also make clear that these experiments evaluate performance on held-out samples, not discretization invariance in the uniform-grid sense.

5.2.1 Cylinder flow

Discuss why the cylinder flow task is a meaningful non-uniform benchmark. Mention the varying geometry, varying point clouds, and the fact that the model must predict a vector field on an irregular spatial domain.

Discuss the error structure in the representative prediction. The largest source of error now is ”steady state”. not all samples appear to have reached a comparable steady or quasi-steady state by the final time. This is a dataset limitation, not model limitation. Some samples possess a tail that oscillates about a mean position at the final time. This is very hard for the ”one shot” (I dont remember what its called, but a model that predicts the final state directly from the initial state instead of an auto rollout approach) geo-FNO to learn. This made the hyperparameter sweep quite difficult, as it was not expected for the model to manage to capture the transient behavior of the samples that had not reached a steady state, and the loss was very high for those samples.

Other things to discuss? discuss where in the plot the loss is high. other?

5.2.2 HTGR column temperature dataset

The strong performance on this dataset should be interpreted in the context of the task. The HTGR dataset is smoother and lower-dimensional than the cylinder flow benchmark: the input consists only of the boundary temperature and axial level[1], and the mesh is shared across all samples. The result therefore demonstrates that geo-FNO can be applied to reactor-relevant non-uniform data, but it does not by itself establish performance on more complex nuclear engineering datasets.

refer again to Appendix C?

5.2.3 Role of geo-FNO

Discuss what the non-uniform results do and do not show. They show that geo-FNO can be applied directly to irregular spatial representations and produce meaningful predictions on held-out data. They do not show discretization invariance in the same sense as the uniform-grid experiments, because no systematic variation in point-cloud density or mesh resolution was tested.

This is also the place to mention whether the learned coordinate mapping ϕ is useful to show. If ϕ helps explain the result or a failure mode, include it as a separate analysis figure. If it is not discussed directly, it is better left out of the main text or moved to an appendix.

is this section useful? is there anything else to talk about in this chapter? other things to discuss?

[1] As described in appendix C, a trigonometric representation of z and T was found to exhibit the lowest loss and was therefore used for the HTGR results reported in the main text.

Chapter 6

Conclusions and Future Work

This chapter gives an overview of the project's contributions. After this overview, we will reflect on the results and draw some conclusions. Finally, some ideas for future work will be discussed.

6.1 Contributions and Conclusions

showed that CNO is more robust to resolution changes than FNO, and connected this to the bandlimited assumption and the way that Fourier truncation does not preserve the same physical meaning across resolutions. Showed that geo-FNO can be applied to non-uniform meshes and performs well on the irregular-domain datasets, but also identified a seam artifact that appears across datasets and can corrupt low-k_x modes. Showed that geo-FNO can be applied to nuclear datasets.

split into two sections I think; decide structure once rest of thesis is written.

6.2 Future work

talk about possible improvements, shortcomings, what I would have done differently knowing what i know now etc.

to mention:

- I did not investigate "discretization invariance" for non uniform meshes. here, this would mean something like worse sampling, fewer data points, or a subsampled section of the domain. i will have explained in the thesis that standard discretization invariance is not really applicable to non-uniform meshes, but I could have done some experiments to see how the models perform under these conditions.

- I would have liked to have included another architecture for non uniform meshes. for uniform grids, having the global FNO and local CNO proved to be a good contrast, but for non uniform meshes, I only had the geo-FNO. it would have been interesting to have a local architecture for non uniform meshes as well, to see if the same trends

hold as for uniform meshes. This especially holds for the cylinder flow dataset, as a local model is likely to perform better than the geo fno on such a dataset.

- I would have liked to have performed more actual experiments for non uniform meshes. For example to compare geo fno to interpolating to a regular grid and using FNO, or to compare geo fno to a local architecture on the cylinder flow dataset. I would have also liked to have done some experiments to investigate the seam artifact that appears in geo fno predictions, and see if this can be mitigated through architectural changes or training strategies.

- I would have liked to try to make a geo-CNO. I imagine taking the coordinate transform IPHI from geo-FNO, producing a regular grid and feeding that into a CNO. This could then already include the resampling onto a fixed internal grid. This would be interesting to see if the bandlimited assumption can also help stabilize learning on non uniform meshes, and if the local inductive bias of CNO can help mitigate the seam artifact that appears in geo-FNO. This was also mentioned in the CNO paper as a potential future direction, but I did not have time to explore it.

- The fact that the HTGR dataset is not very challenging, and how a more complex dataset could be constructed to better test the capabilities of the models.

- I would have liked to investigate arbitrary time prediction for these models. Ex. look into autoregressive rollout and stabilizing that. Could that have made the geo-FNO able to predict the final time oscillatory behavior of some cylinder flow samples?

Bibliography

[1] Francesca Bartolucci, Emmanuel de Bézenac, Bogdan Raonic, Roberto Molinaro, Siddhartha Mishra, and Rima Alaifari. Representation equivalent neural operators: a framework for alias-free operator learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 69661–69672. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/dc35c593e61f6df62db541b976d09dcf-Paper-Conference.pdf.

[2] G. K. Batchelor. *An Introduction to Fluid Dynamics*. Cambridge University Press, Cambridge, 1967.

[3] Gary J. Chandler and Rich R. Kerswell. Invariant recurrent solutions embedded in a turbulent two-dimensional kolmogorov flow. *Journal of Fluid Mechanics*, 722:554–595, 2013. doi: 10.1017/jfm.2013.122.

[4] Elliot J. Chikofsky and James H. Cross II. Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17, 1990. doi: 10.1109/52.43044.

[5] Alexandre J. Chorin and Jerrold E. Marsden. *A Mathematical Introduction to Fluid Mechanics*. Springer, New York, 3 edition, 1993.

[6] Aditi Gulati, Armin Felahatpisheh, and Camilo E. Valderrama. Feature engineering through two-level genetic algorithm. *Machine Learning with Applications*, 21:100696, 2025. ISSN 2666-8270. doi: https://doi.org/10.1016/j.mlwa.2025.100696. URL https://www.sciencedirect.com/science/article/pii/S2666827025000799.

[7] Jan Hagnberger, Daniel Musekamp, and Mathias Niepert. Calm-pde: Continuous and adaptive convolutions for latent space modeling of time-dependent pdes, 2025. URL https://arxiv.org/abs/2505.12944.

[8] Nikola B. Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces. *CoRR*, abs/2108.08481, 2021. URL https://arxiv.org/abs/2108.08481.

[9] Zongyi Li, Nikola B. Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *CoRR*, abs/2010.08895, 2020. URL https://arxiv.org/abs/2010.08895.

[10] Abdou Njifenjou. Some models of darcy flows in porous media, 07 2023.

[11] Alan V. Oppenheim, Alan S. Willsky, and Syed Hamid Nawab. *Signals and systems*. Pearson Education Limited, 2014.

[12] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks, 2020. URL https://arxiv.org/abs/2002.09405.

[13] Alberto Vela-Martín and Marc Avila. Large-scale patterns set the predictability limit of extreme events in kolmogorov flow. *Journal of Fluid Mechanics*, 986:A2, 2024. doi: 10.1017/jfm.2024.263.

[14] Colin Wheildon. *Type and Layout: How Typography and Design Can Get Your Message Across, or Get in the Way*. Strathearn Communication Consultants, Sydney, 1995.

Appendix A

Disclosure on AI use

AI-based tools were used during this thesis project as support for coding, debugging, troubleshooting, source finding, and comprehension of technical material. During the writing process, they were also used to help structure sections and improve phrasing.

All AI-generated suggestions were reviewed and edited before being included. The research decisions, experiments, interpretation of results, and final thesis text remain my own responsibility.

Appendix B

Navier-Stokes data generation details

This appendix provides additional details on the generation of the self-produced Navier-Stokes dataset used in the uniform grid experiments. The dataset is formulated in vorticity form on the unit square and is generated numerically from the incompressible Navier-Stokes equation [2, 5]:

$$\partial_t \omega + u \cdot \nabla \omega = \nu \Delta \omega + f, \tag{B.1}$$

where ω denotes the vorticity field, u the incompressible velocity field, ν the viscosity, and f an external forcing term. The velocity field is not part of the stored dataset or the prediction target. Instead, it is reconstructed from the vorticity during data generation in order to evaluate the nonlinear advection term, using the streamfunction relation [2, 5]:

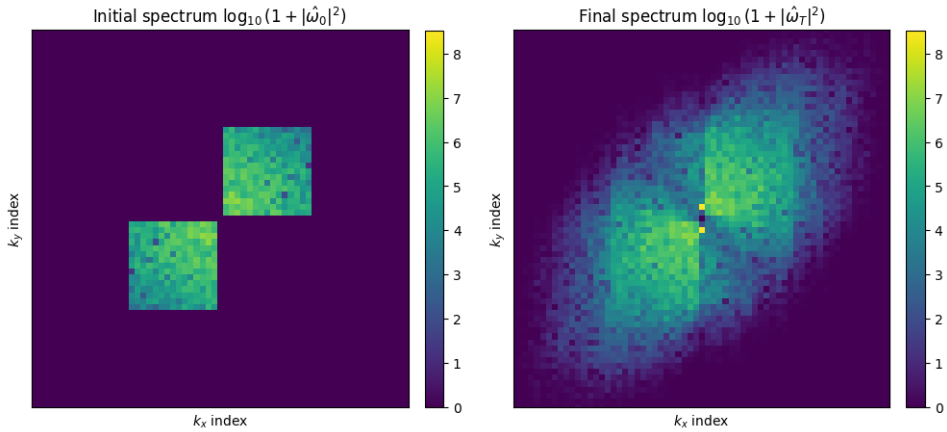

Figure B.1: Fourier spectra of an example initial and final vorticity field from the generated Navier-Stokes dataset. The initial spectrum reflects the truncated low-frequency initialization with Gaussian weighting, while the final spectrum shows the redistribution of spectral energy over time.

$$-\Delta\psi = \omega, \qquad u = \nabla^{\perp}\psi. \tag{B.2}$$

In all generated samples, the viscosity is kept fixed at $\nu = 10^{-5}$. The forcing term is also shared across samples and is given by:

$$f(y,t) = 0.5\,(1 + 0.5\sin(0.5t))\sin(4\pi y), \tag{B.3}$$

which is consistent with the Kolmogorov-flow literature, where the forcing is commonly chosen as a single sinusoidal mode that varies only in one spatial direction [3, 13]. In the present dataset, the spatial factor $\sin(4\pi y)$ serves as the corresponding single-mode forcing on the unit square. The prefactor was chosen so that the forcing remains smooth, bounded, and of moderate magnitude throughout the simulation: specifically, $0.5(1+0.5\sin(0.5t)) \in [0.25, 0.75]$. The additional temporal modulation is not intended to reproduce a specific benchmark forcing law, but was introduced as a synthetic extension to generate trajectories with mild time-dependent variation in the energy input while avoiding abrupt or high-frequency changes.

The dataset contains 1000 samples at a resolution of 256×256. For each sample, an initial vorticity field is generated in Fourier space. Fourier coefficients are assigned for indices $(k_x, k_y) = (i, j)$ with $i, j \in \{1, \ldots, k_{\max} - 1\}$ and $k_{\max} = 16$. These coefficients are random complex numbers whose amplitudes are modulated by a Gaussian envelope of the form $\exp\left(-\frac{1}{2}\frac{i^2+j^2}{(k_{\max}/2)^2}\right)$, which suppresses higher frequencies and yields smooth initial conditions. The coefficients in the $-k_x, -k_y$ quadrant are then set by conjugate symmetry so that the inverse Fourier transform produces a real-valued field. No coefficients are assigned in the mixed-sign quadrants. The initial vorticity field is finally obtained by applying an inverse Fourier transform, followed by normalization such that the maximum absolute value of the vorticity is 1.

Each initial condition is then evolved forward in time using a fourth-order Runge–Kutta (RK4) integration scheme to obtain the corresponding later vorticity field stored in the dataset.

This initialization procedure produces smooth but diverse initial conditions while concentrating most spectral energy in the low-frequency region. Figure B.1 illustrates this by showing the Fourier spectra of an example initial state and its corresponding final state. The figure highlights both the initially active modes and the effect of the Gaussian envelope, as well as the redistribution of spectral energy under the Navier-Stokes dynamics.

Appendix C

HTGR input engineering details

This appendix provides additional details on the input engineering used for the HTGR column temperature dataset, as introduced in Chapter 3. In this dataset, the model input consists only of the axial level z and the boundary temperature T. Since this input is low-dimensional, several alternative representations were tested to determine whether a more expressive encoding of the same information would improve predictive performance.

Table C.1 shows a representative subset of the tested input representations. The raw representation (z, T) already performs reasonably well, with a test relative RMSE of 3.75×10^{-3}. Replacing these raw variables with min–max normalized variables,

$$\hat{z} = \frac{z - z_{\min}}{z_{\max} - z_{\min}}, \qquad \hat{T} = \frac{T - T_{\min}}{T_{\max} - T_{\min}},$$

scales both inputs to the range $[0, 1]$. This improves the test relative RMSE to 2.57×10^{-3} and reduces the test MSE by approximately a factor of two. Thus, for this dataset, giving the model inputs on a common bounded scale is beneficial.

The next representation augments the min–max normalized variables with quadratic and interaction terms, giving $(\hat{z}, \hat{T}, \hat{z}^2, \hat{T}^2, \hat{z}\hat{T})$. This further reduces the test relative RMSE to 2.44×10^{-3}, although the improvement over the basic min–max normalized representation is modest. This is consistent with the fact that $\hat{z}^2$ and $\hat{T}^2$ are not physically meaningful variables for this problem. Instead, they provide a simple polynomial expansion of the two scalar inputs, making some nonlinear dependencies easier for the model to represent.

Finally, a trigonometric representation was tested using the min–max normalized variables together with boundary-aware functions of the axial coordinate: $(\hat{z}, \hat{T}, \sin(\pi(z - z_{\min})/L_z), \cos(\pi(z - z_{\min})/L_z), \hat{z}\hat{T})$, where $L_z = z_{\max} - z_{\min}$. This choice is more directly motivated by the structure of the data. In particular, the sine term vanishes at the axial boundaries, which reflects the observed trend that the peak temperature varies smoothly with z. The cosine term provides a complementary axial feature, and the interaction term $\hat{z}\hat{T}$ gives the model direct access to coupled dependence on axial position and boundary temperature.

The trigonometric representation gives the best performance among the tested configurations, with a test relative RMSE of 2.07×10^{-3}. This shows that using physical understanding of the dataset in the input representation can improve the Geo-FNO prediction

Table C.1: Performance comparison of selected input representations for the HTGR column temperature dataset.

Input representation	Val relRMSE	Test relRMSE / MSE
(z, T)	3.334622×10^{-3}	3.752964×10^{-3} / 7.120657×10^{0}
min–max normalized $(\hat{z}, \hat{T})$	3.124016×10^{-3}	2.573528×10^{-3} / 3.468112×10^{0}
min–max normalized $(\hat{z}, \hat{T}, \hat{z}^2, \hat{T}^2, \hat{z}\hat{T})$	2.925913×10^{-3}	2.443143×10^{-3} / 2.997041×10^{0}
$(\hat{z}, \hat{T}, \sin(\pi(z - z_{\min})/L_z), \cos(\pi(z - z_{\min})/L_z), \hat{z}\hat{T})$	2.105979×10^{-3}	2.074195×10^{-3} / 2.180137×10^{0}

accuracy. The trigonometric representation was therefore used for the HTGR results reported in the main text.

Appendix D

Hyperparameters

This appendix provides a complete list of the hyperparameters used for each model and dataset in the experiments described in Chapter 3. The hyperparameters include architectural choices, training settings, and any dataset-specific parameters that were relevant for model performance. The listed hyperpameters were determined through a combination of prior literature, preliminary experiments, and manual tuning to achieve good performance.

Appendix E

Geo-FNO bug fix

insert figure of same sample from cylinder flow a) without iphi b) with iphi c) after fix, with iphi

insert figure of iphi on cylinder dataset before and after bug fix

During testing of the Geo-FNO, a central, vertical, seam-like artifact appeared across all tested datasets. An example of this is shown in figure **??** Inspecting the coordinate mapping iphi learned revealed that it split the domain down the middle, as seen in figure **??**iphi before and after). Training a model without iphi lead to worse performance, meaning the split observed in iphi is to minimize the number of points sampled in the area where the seam appeared and decreased performance. Importantly, iphi itself was not the source of the seam. (how much of my troubleshooting should i talk about? probably very little. i have several plots i used to narrow down what was causing the issue, but i do not think that is relevant. i will just state what the bug was, why that was wrong, how it was fixed and the performance increase we saw as a result.)

the bug was found to be caused by a mismatch in the reconstruction of discarded modes. the geo-FNO keeps N modes in each direction, corresponding to N positive and N negative modes in each direction. (include a figure of the modes in 2d for clarity? not necessary for explanation but might help reader understand) However, the real valued field constraint/Hermitian symmetry can be used to reduce the number of modes kept. it is given as U(-kx, -ky) = conj(U(kx, ky)). as such, the negative ky modes are explicitly discarded to save memory, as any mode with negative ky can be reconstructed using **??**. separate learned complex weights are applied to (+kx, +ky) and (-kx, +ky) respectively. when reconstructing the physical field, the real valued constraint is used to reconstruct the discarded ky modes. the seam was caused by a bug in how these modes were matched up. the exact line where this reconstruction was done was this: (insert exact line from original geo fno repo before my pull). to match the modes correctly in the code, the lists of kx and ky modes are flipped. For the ky modes, the flip is straightforward and works as intended. the kx modes, however, are stored as [0, 1, ..., N-1, -N, -(N-1), ..., -1]. the 0th mode is self conjugate, as is the -N mode referred to as the Nyquist edge mode. a simple flip as is applied in this line of code gives [-1, -2, ..., -N, N-1, N-2, ..., 0]. this leads to an incorrect matching that can be expressed as: U(kx, ky) = conj(U(-(kx + 1), -ky)) for kx ¿ 0. This mismatch is the greatest for kx = 0 and the other low kx modes, leading to corruption that causes the vertical seam.

fixing this line of code such that the modes are correctly matched lead to a decrease in MSE loss for the cylinder flow dataset from 5.5e-3 to 4.7e-3, or a 15% decrease. Visually, this fix removed the seam entirely, as can be seen in figure **??** The authors of the geo-FNO paper were notified of these findings, and my proposed fix has been implemented in the official repository.

Appendix F

Requirements and Guidelines

This chapter details some requirements and guidelines for MSc theses submitted to the Software Engineering Research Group.

F.1 Requirements

F.1.1 Layout

- Your thesis should contain the formal title pages included in this document (the page with the TU Delft logo and the one that contains the abstract, student id and thesis committee). Usually there is also a cover page containing the thesis title and the author (this document has one) but this can be omitted if desired.
- Base font should be an 11 point serif font (such as Times, New Century Schoolbook or Computer Modern). Do not use sans-serif fonts such as Arial or Helvetica. *Sans-serif type is intrinsically less legible than seriffed type* [14].
- The final thesis and drafts submitted for reviewing should be printed double-sided on A4 paper.

F.1.2 Content

- The thesis should contain the following chapters:
  - Introduction.
    Describes project context, goals and your research question(s). In addition it contains an overview of how (the remainder of) your thesis is structured.
  - One or (usually) more "main" chapters.
    Present your work, the experiments conducted, tool(s) developed, case study performed, etc.
  - Overview of Related Work
    Discusses scientific literature related to your work and describes how those approaches differ from what you did.

- Discussion/Evaluation/Reflection
  What went well, what went less well, what can be improved?
- Conclusions, Contributions, and (Recommendations for) Future Work
- Bibliography

F.1.3 Bibliography

- Make sure you've included all required data such as journal, conference, publisher, editor and page-numbers. When you're using BIBTEX, this means that it won't complain when running `bibtex your-main-tex-file`.
- Make sure you use proper bibliographic references. This especially means that you should avoid references that **only** point at a website and not at a printed publication.

  For example, it's OK to add a URL with the entry for an article describing a tool to point at its homepage, but it's not OK to just use the URL and not mention the article.

F.2 Guidelines

- The main chapters of a typical thesis contain approximately 50 pages.
- A typical thesis contains approximately 50 bibliographic references.
- Make sure your thesis structure is balanced (check this in the table of contents).

  Typically the main chapters should be of equal length. If they aren't, you might want to revise your structure by merging or splitting some chapters/sections.

  In addition, the (sub)section hierarchies with the chapters should typically be balanced and of similar depth. If one or more are much deeper nested than others in the same chapter this generally signals structuring problems.
- Whenever you submit a draft of your thesis to your supervisor for reviewing, make sure that you have checked the spelling and grammar. Moreover, *read it yourself at least once from start to end, before submitting to your supervisor*.

  Your supervisor is not a spelling/grammar checker!
- Whenever you submit a second draft, include a short text which describes the changes w.r.t. the previous version.